Databricks ML Associate
100 Q&A Rapid Review

Exam Prep Deck

How to use this deck

Odd slide: the exam-style question.

Next slide: correct answer + **why the other options are wrong**.

Recommendation: cover the answer slide, answer aloud, then reveal.

Q1 — MLflow Tracking

A data scientist needs parameters, metrics, and the model artifact logged automatically for a scikit-learn experiment. Best approach?

A. Manually write values to a CSV.

B. Use `mlflow.autolog()` or explicit `log_param`/`log_metric` within a run.

C. Print values to the notebook output.

D. Save a screenshot of the experiment.

A1 — Answer: B

Correct: B.

`mlflow.autolog()` captures params, metrics, and model artifacts for supported libraries in one line.

A and D are manual and not reproducible.

C is not tracked in the registry.

Q2 — Model Registry

A model must be promoted from development to production with a clear, auditable lifecycle. Recommended Databricks approach?

A. Email the pickle file to the production team.

B. Register in Unity Catalog with versions and an alias such as `@champion`.

C. Rename the artifact folder.

D. Copy the model file to a shared DBFS path.

A2 — Answer: B

Correct: B.

Unity Catalog registry gives governance, lineage, versioning, and aliases.

`@champion` is the modern replacement for old "Production" stage.

A, C, D have no audit trail and no governance.

Q3 — Feature Store

A team wants to avoid training-serving skew. Which Databricks capability is designed for this?

A. A shared Excel spreadsheet of feature definitions.
B. Unity Catalog Feature Store with point-in-time joins.
C. Recomputing features separately in serving code.
D. Using only raw columns at inference.

A3 — Answer: B

Correct: B.

Feature Store = single source of truth for features used in training and serving.

Point-in-time joins prevent future-data leakage.

C is exactly what creates training-serving skew.

Q4 — AutoML

A data scientist wants a quick baseline on a labeled tabular dataset. Most appropriate feature?

A. Databricks AutoML.
B. Building a custom neural network from scratch.
C. Deploying an endpoint before training.
D. Creating an online feature table only.

A4 — Answer: A

Correct: A.

AutoML produces baseline models, leaderboard, and generated notebooks.

B is slow; C is out of order; D doesn't train anything.

Q5 — ML Runtime

Primary advantage of Databricks Runtime for Machine Learning?

A. Includes common ML libraries and integrations preinstalled.
B. Eliminates all package conflicts automatically.
C. Removes the need for cluster permissions.
D. Allows skipping feature engineering entirely.

A5 — Answer: A

Correct: A.

ML Runtime ships with MLflow, Hyperopt, sklearn, XGBoost, etc.

B is false — conflicts can still occur.

C and D are obviously wrong.

Q6 — Unity Catalog Registry

Key benefit of registering models in Unity Catalog vs workspace registry?

A. Models become accessible only to the original author.
B. Models are governed, discoverable, and promotable across workspaces at account level.
C. Models no longer need a version number.
D. Models are automatically deployed to serving endpoints.

A6 — Answer: B

Correct: B.

UC = account-level governance and cross-workspace discovery.

A restricts access (opposite of intent).

C is false — versioning always exists.

D is false — deployment is separate.

Q7 — MLflow Search

How to programmatically find the best run by a metric?

A. Use `mlflow.search_runs()` and sort the DataFrame.
B. Manually click through each run in the UI.
C. Export all runs to CSV and use Excel.
D. Delete runs until one remains.

A7 — Answer: A

Correct: A.

`search_runs()` returns a pandas DataFrame; sort by metric.

B, C are manual; D is destructive.

Q8 — Model Aliases

What replaced older stages (Staging/Production/Archived) in Unity Catalog?

A. Numeric version numbers only.
B. Aliases such as `champion` and `challenger`.
C. Tags only.
D. File system paths.

A8 — Answer: B

Correct: B.

Aliases are mutable pointers to a specific model version.

E.g., `models:/my_model@champion`.

C (tags) are static metadata — not pointers.

Q9 — Online vs Offline Features

Primary difference between online and offline feature tables?

A. Online tables are for training; offline for serving.
B. Online tables serve low-latency inference; offline for batch training/analysis.
C. Online tables cannot be created in Unity Catalog.
D. Offline tables do not support point-in-time joins.

A9 — Answer: B

Correct: B.

Online = low-latency lookup (e.g., DynamoDB-backed).

Offline = large-scale batch / training reads.

A reverses the roles; C and D are false.

Q10 — Feature Engineering Client

Which class creates/manages feature tables in Unity Catalog?

A. `FeatureEngineeringClient`.
B. `SparkSession`.
C. `MLflowClient`.
D. `Pipeline`.

A10 — Answer: A

Correct: A.

`FeatureEngineeringClient` → `create_table`, `write_table`, `create_training_set`.

`MLflowClient` manages runs/registry, not feature tables.

Q11 — AutoML Output

What useful artifact does AutoML generate for review and customization?

A. A screenshot of the leaderboard only.
B. Generated notebooks for data exploration and best trial.
C. A compiled binary with no source code.
D. A cluster event log.

A11 — Answer: B

Correct: B.

AutoML generates a data exploration notebook and a best-trial training notebook. These can be opened, edited, and re-run as a starting point.

Q12 — MLOps Best Practice

Which setup best reflects sound MLOps strategy?

A. One shared workspace where notebooks are edited directly in production.
B. Separate dev/staging/prod environments with versioned code and tracked artifacts.
C. Manual model file copies between users.
D. Training models without keeping prior run metadata.

A12 — Answer: B

Correct: B.

Environment separation + version control + experiment tracking = repeatable, auditable.

A is the classic "everyone edits prod" antipattern.

Q13 — Reproducibility

Which practice most directly improves reproducibility?

A. Keeping training code in source control and logging params/artifacts in MLflow.
B. Relying on notebook revision history alone.
C. Letting each data scientist rename columns locally.
D. Saving only final predictions.

A13 — Answer: A

Correct: A.

Reproducibility = code + params + artifacts all pinned and logged.

Notebook history alone misses data & environment state.

Q14 — Model Promotion Strategy

When is promoting code preferred over promoting models?

A. When the model is a simple scikit-learn model.
B. When the training pipeline is complex and the model artifact is small.
C. When the model artifact is large and training is expensive.
D. When the team uses only AutoML.

A14 — Answer: B

Correct: B.

Promote-code: rebuild the model in each environment from versioned code — cleaner for complex pipelines with cheap training.

Promote-model: ship the artifact — better when training is expensive (C).

Q15 — MLflow UI

Where in the MLflow UI can a user find the notebook that was run to create a run?

A. The “Source” link for the run.
B. The “Artifacts” tab only.
C. The cluster event log.
D. The Databricks home page.

A15 — Answer: A

Correct: A.

"Source" links back to the notebook and revision used for that run.

Artifacts tab only shows stored files.

Q16 — Feature Store Creation

Which method creates a feature table in Unity Catalog?

A. `fe.create_table()`.
B. `spark.sql("CREATE TABLE")`.
C. `mlflow.log_artifact()`.
D. `dbutils.fs.mkdirs()`.

A16 — Answer: A

Correct: A.

`FeatureEngineeringClient.create_table()` registers schema, primary keys, timestamp keys. Plain `CREATE TABLE` does not register as a feature table.

Q17 — Point-in-Time Lookup

Why is point-in-time lookup important for feature-store training sets?

A. Ensures training does not use feature values unavailable at prediction time.
B. It speeds up the query.
C. It automatically imputes missing values.
D. It converts categorical features to numeric.

A17 — Answer: A

Correct: A.

Point-in-time join fetches each entity's feature values as of the label timestamp.

Prevents label leakage from future data.

Q18 — Model Tags

How can a tag be set on a registered model in Unity Catalog?

A. `mlflow.set_model_tag()`.
B. `spark.conf.set()`.
C. `dbutils.notebook.exit()`.
D. `display()`.

A18 — Answer: A

Correct: A.

`mlflow.set_model_tag()` and `mlflow.set_model_version_tag()` handle model/version tags. Others are unrelated APIs.

Q19 — MLflow Run Identification

How to identify the best run using the MLflow Client API?

A. `client.search_runs()`.
B. `client.list_experiments()`.
C. `client.get_run()`.
D. `client.delete_run()`.

A19 — Answer: A

Correct: A.

`search_runs()` supports filters like `metrics.accuracy > 0.9` and ordering.

`get_run()` needs a run_id you already know.

Q20 — Champion Promotion

How is a challenger promoted to champion using aliases?

A. Delete the old model and upload the new one.
B. Move the `@champion` alias to the new model version.
C. Rename the model.
D. Create a new experiment.

A20 — Answer: B

Correct: B.

Aliases are mutable pointers — reassign `@champion` to new version.

This is zero-downtime: consumers keep using `@champion`.

Q21 — MLflow Experiment Organization

A data scientist wants to track hyperparameter tuning with many child runs. Best MLflow pattern?

A. Separate experiment for each trial.
B. Start child runs inside the parent run using `mlflow.start_run(nested=True)`.
C. Log all trials to one run without nesting.
D. Avoid logging parameters.

A21 — Answer: B

Correct: B.

Nested runs group all hyperparameter trials under one parent.

Easier to compare trials and identify the best config.

Q22 — Workspace vs UC Registry

Limitation of the workspace MLflow Model Registry vs Unity Catalog?

A. Workspace models cannot be versioned.
B. Workspace models are scoped to a single workspace and lack account-level governance.
C. Workspace models cannot be loaded in notebooks.
D. Workspace models cannot be logged with MLflow.

A22 — Answer: B

Correct: B.

Workspace registry lives inside one workspace — no cross-workspace promotion.

UC registry is account-wide and governed.

Q23 — MLflow Model Loading

Which function loads a logged MLflow model for inference?

A. `mlflow.pyfunc.load_model()`.
B. `mlflow.register_model()`.
C. `mlflow.log_model()`.
D. `mlflow.start_run()`.

A23 — Answer: A

Correct: A.

`mlflow.pyfunc.load_model()` or flavor-specific `load_model()` loads for scoring.

`log_model()` logs, `register_model()` registers — neither loads for scoring.

Q24 — Online Table Sync

How are online feature tables kept synchronized with offline tables?

A. Manual export/import.
B. Databricks automatically publishes online tables from offline tables.
C. Online tables are never synchronized.
D. Only batch jobs can synchronize them.

A24 — Answer: B

Correct: B.

Databricks automatically syncs offline $\rightarrow$ online.

This is part of the Feature Store design to reduce serving latency.

Q25 — AutoML Constraints

Which constraint can be specified when running Databricks AutoML?

A. A time limit for the experiment.
B. A specific random seed only.
C. A requirement to use only neural networks.
D. A fixed number of features.

A25 — Answer: A

Correct: A.

`timeout_minutes` caps total AutoML runtime.

`exclude_frameworks` can exclude specific algorithms — but no "only NN" option.

Q26 — MLflow Run Deletion

Which API call deletes an MLflow run?

A. `client.delete_run(run_id)`.
B. `client.get_run(run_id)`.
C. `client.search_runs()`.
D. `mlflow.end_run()`.

A26 — Answer: A

Correct: A.

`delete_run()` marks a run as deleted.

`end_run()` only terminates an active run — it does not delete it.

Q27 — Production Candidate Marking

Recommended way to mark a model version as current production candidate in UC?

A. Set the `@champion` alias on that version.
B. Delete all other versions.
C. Rename the model to include “prod”.
D. Create a new schema.

A27 — Answer: A

Correct: A.

`@champion` is the intended mechanism.

Deleting versions destroys history; renaming is not portable.

Q28 — Experiment Artifacts

Where are MLflow run artifacts stored by default in Databricks?

A. In local cluster disk only.
B. In the MLflow experiment's artifact location (DBFS or cloud storage).
C. In notebook revision history.
D. In the driver's memory.

A28 — Answer: B

Correct: B.

Each experiment has an `artifact_location` (DBFS or cloud URI).

Local disk is ephemeral; revision history stores notebook code.

Q29 — Registry Permissions

What permission is required to promote a model version to champion in UC?

A. `USE CATALOG` only.

B. `EXECUTE` on the model.

C. `MANAGE` or `APPLY TAG` on the model.

D. `SELECT` on the training table.

A29 — Answer: C

Correct: C.

Aliases/tags are metadata edits → need `MANAGE` or `APPLY TAG`.

`SELECT` on the training table doesn't grant model metadata rights.

Q30 — Autologging Scope

Which libraries does `mlflow.autolog()` support in Databricks?

A. Only scikit-learn.
B. Multiple including scikit-learn, XGBoost, LightGBM, and Spark ML.
C. Only TensorFlow.
D. Only PyTorch.

A30 — Answer: B

Correct: B.

Databricks autologging supports sklearn, XGBoost, LightGBM, Spark ML, TensorFlow, PyTorch, and more.

Q31 — Feature Store Lookup

Which method retrieves features from a feature table for inference?

A. `fe.get_online_features()`.
B. `spark.read.table()` only.
C. `mlflow.load_model()`.
D. `dbutils.fs.head()`.

A31 — Answer: A

Correct: A.

`get_online_features()` returns low-latency feature records for a list of entity IDs.

`spark.read.table()` reads batch features — not for real-time.

Q32 — Model Signature Enforcement

What happens when a model with signature is served and input does not match?

A. The request fails validation.
B. The model silently ignores the mismatch.
C. The model retrains automatically.
D. The signature is updated.

A32 — Answer: A

Correct: A.

Signatures are enforced at inference; invalid schema → HTTP error.

This is the whole point: catch serving-time schema drift.

Q33 — Tracking Server Location

In Databricks, where are MLflow experiments and runs stored?

A. On driver's local disk.
B. In the Databricks-managed MLflow tracking service.
C. In an external Git repository.
D. In the cluster event log.

A33 — Answer: B

Correct: B.

Databricks provides a managed tracking server — no local setup needed.

Workspace is the default tracking server.

Q34 — Deployment Readiness

Which check is most important before promoting to production?

A. Only training accuracy.
B. Performance on a held-out test set and validation of input schema.
C. Number of notebook cells.
D. Cluster size used for training.

A34 — Answer: B

Correct: B.

Production readiness = good generalization + correct input schema.

Training accuracy alone overestimates performance.

Q35 — Feature Table Primary Keys

Purpose of primary keys in a UC feature table?

A. Uniquely identify each entity for point-in-time lookups.
B. Encrypt the table.
C. Reduce storage costs.
D. Replace the timestamp column.

A35 — Answer: A

Correct: A.

Primary keys + timestamp keys = join keys for PIT lookups.

They do not encrypt or replace timestamps.

Q36 — MLflow Run Hierarchy

Which structure correctly represents experiments/runs/artifacts?

A. Experiments contain runs; runs contain artifacts.
B. Runs contain experiments; experiments contain artifacts.
C. Artifacts contain runs; runs contain experiments.
D. Experiments and runs are unrelated.

A36 — Answer: A

Correct: A.

Experiment → many runs → each run logs artifacts, params, metrics.

Q37 — AutoML Explainability

What does AutoML provide to help interpret the best model?

A. SHAP-based feature importance and generated notebooks.
B. Only a leaderboard.
C. Only final predictions.
D. A compiled binary.

A37 — Answer: A

Correct: A.

AutoML leaderboard provides SHAP summaries and per-row explanations.

Notebooks make the winning pipeline reproducible.

Q38 — Registry Lineage

What information does Unity Catalog retain for lineage?

A. Only the model name.

B. Run ID, source experiment, and model version lineage.

C. Only file size.

D. Only upload timestamp.

A38 — Answer: B

Correct: B.

Lineage links model version → run → experiment → input tables.

This enables audit and impact analysis.

Q39 — Regression Metrics

RMSE = 25, MAE = 20, $R^2 = 0.85$. Which two statements are correct?

A. Model captures 85% of variance.

B. RMSE ¿ MAE suggests larger errors exist.

C. Model is underfitting.

D. MAE is 25.

A39 — Answer: A and B

Correct: A, B.

$R\hat{2} = 0.85$ → 85% of variance explained.

RMSE penalizes large errors more than MAE RMSE ¿ MAE some large errors.

D is wrong — MAE is 20, not 25.

Q40 — Imbalanced Classification

98% accuracy, 2% positives. What should be done?

A. Ship immediately.

B. Evaluate precision, recall, F1, PR-AUC; consider class weights or resampling (training only).

C. Increase epochs.

D. Remove positive class.

A40 — Answer: B

Correct: B.

Accuracy is misleading on imbalanced classes.

Use PR-AUC, F1, recall; resample or reweight on train only.

Never resample test set (would inflate metrics).

Q41 — One-Hot Encoding

When is one-hot encoding least appropriate?

A. Low cardinality.
B. Extremely high cardinality.
C. Using linear models.
D. Feature is a string.

A41 — Answer: B

Correct: B.

High cardinality → too many sparse columns → memory blow-up.

Prefer target encoding or feature hashing.

Q42 — Imputation

For a highly skewed distribution, preferred imputation strategy?

A. Mean.
B. Median.
C. Mode.
D. Zero.

A42 — Answer: B

Correct: B.

Median is robust to skew and outliers.

Mean gets pulled by skew and large values.

Q43 — Log Transformation

When is a log transformation most appropriate?

A. Normally distributed feature.
B. Right-skewed feature spanning orders of magnitude.
C. Categorical feature.
D. Binary target.

A43 — Answer: B

Correct: B.

Log compresses large values, reduces skew, stabilizes variance.

Not meaningful for already-normal or categorical features.

Q44 — Estimator vs Transformer

Key difference between an Estimator and a Transformer in Spark ML?

A. Estimator has `fit()`; Transformer has `transform()`.
B. Estimator transforms data; Transformer fits models.
C. They are identical.
D. Transformer cannot be used in a Pipeline.

A44 — Answer: A

Correct: A.

Estimator.fit(df) $\rightarrow$ Transformer.

Transformer.transform(df) $\rightarrow$ df.

Both are Pipeline stages.

Q45 — Spark ML Pipeline

Why wrap preprocessing + training in a Spark ML Pipeline?

A. Ensures same preprocessing at inference as training.
B. Reduces the number of columns.
C. Avoids using MLflow.
D. Eliminates the need for a test set.

A45 — Answer: A

Correct: A.

PipelineModel captures fitted preprocessing parameters.

Prevents train/serve skew.

Q46 — Hyperopt Parallelization

Which Hyperopt component distributes trials across a Spark cluster?

A. `SparkTrials`.
B. `Trials`.
C. `fmin`.
D. `hp.uniform`.

A46 — Answer: A

Correct: A.

`SparkTrials` → runs trials in parallel on Spark workers.

`Trials` is single-node serial.

Q47 — Hyperopt Algorithm

Which algorithm does Hyperopt use via `tpe.suggest`?

A. Grid search.
B. Random search.
C. Tree-structured Parzen Estimator (Bayesian optimization).
D. Exhaustive search.

A47 — Answer: C

Correct: C.

TPE = sequential model-based (Bayesian) optimizer.

Smarter than grid/random at the same budget.

Q48 — CV vs Single Split

Main trade-off between cross-validation and a single train-validation split?

A. CV is faster.
B. CV gives a more stable estimate but is more expensive.
C. Single split always more accurate.
D. Same cost.

A48 — Answer: B

Correct: B.

k-fold CV = k model fits $\rightarrow$ more robust but slower.

Single split is noisier but cheap.

Q49 — Imbalanced Metrics

Best metric for a highly imbalanced binary classification task?

A. Accuracy.
B. PR-AUC.
C. MAE.
D. R^2.

A49 — Answer: B

Correct: B.

PR-AUC focuses on minority class performance.

ROC-AUC can look optimistic under heavy imbalance.

Q50 — Random Split

Which Spark operation randomly splits a DataFrame into train and test?

A. `DataFrame.randomSplit()`.
B. `TrainValidationSplit`.
C. `CrossValidator`.
D. `DataFrame.where()`.

A50 — Answer: A

Correct: A.

`df.randomSplit([0.8, 0.2], seed=42)` → list of DataFrames.

`TrainValidationSplit` is for hyperparameter tuning, not raw splitting.

Q51 — StringIndexer

Before one-hot encoding a string column in Spark ML, which stage is typically required?

A. `StringIndexer`.
B. `VectorAssembler`.
C. `StandardScaler`.
D. `Imputer`.

A51 — Answer: A

Correct: A.

`OneHotEncoder` needs numeric index input, produced by `StringIndexer`.

Typical chain: StringIndexer → OneHotEncoder → VectorAssembler.

Q52 — VectorAssembler

Purpose of `VectorAssembler`?

A. Combine multiple feature columns into a vector column.
B. Impute missing values.
C. Encode categorical variables.
D. Split data.

A52 — Answer: A

Correct: A.

Spark ML models require a single "features" vector column.

`VectorAssembler` does that final assembly step.

Q53 — Feature Scaling

Which scaler makes features zero-mean/unit-variance?

A. `MinMaxScaler`.
B. `StandardScaler`.
C. `MaxAbsScaler`.
D. `Normalizer`.

A53 — Answer: B

Correct: B.

`StandardScaler`: $(x - \mu)/\sigma$.

`MinMaxScaler` $\rightarrow [0, 1]$; `Normalizer` $\rightarrow$ unit L-p norm row-wise.

Q54 — Data Leakage

Standardizing before train/test split. What is the risk?

A. Test statistics leak into train $\rightarrow$ overly optimistic evaluation.
B. Underfitting.
C. Features become categorical.
D. Nothing; best practice.

A54 — Answer: A

Correct: A.

Scaler should be fit on train only, then applied to test.

Otherwise leakage inflates metrics.

Q55 — Pandas UDF

Benefit of vectorized pandas UDFs over standard PySpark UDFs?

A. Batch processing (Arrow) $\rightarrow$ much faster.
B. Only work on single-node DataFrames.
C. Require no Python.
D. Auto-handle missing values.

A55 — Answer: A

Correct: A.

Vectorized pandas UDFs use Apache Arrow for batch transfer $\rightarrow$ big speedup.

Standard UDFs are row-by-row (slow).

Q56 — Target Encoding

Target encoding is most useful for which situation?

A. Low-cardinality categorical.
B. High-cardinality categorical.
C. Continuous with outliers.
D. Binary targets only.

A56 — Answer: B

Correct: B.

Replaces category with mean target — compact for many categories.

Use CV/leave-one-out to avoid leakage.

Q57 — Overfitting

Good train performance, poor test performance. What is this called?

A. Underfitting.
B. Overfitting.
C. Data leakage.
D. Regularization.

A57 — Answer: B

Correct: B.

Model memorized training noise $\rightarrow$ poor generalization.

Fix: regularization, more data, simpler model.

Q58 — Bias-Variance

High variance typically corresponds to which problem?

A. Overfitting.
B. Underfitting.
C. Data imbalance.
D. Feature scaling.

A58 — Answer: A

Correct: A.

High variance = very sensitive to training sample = overfit.

High bias = too simple = underfit.

Q59 — Median vs Mean Imputation

When is median imputation preferred?

A. Symmetric feature.
B. Feature with outliers/skew.
C. Categorical feature.
D. No missing values.

A59 — Answer: B

Correct: B.

Median is robust to skew/outliers.

For symmetric, mean is fine.

Q60 — Class Weights

How to mitigate imbalance without resampling?

A. Class weights in training.
B. Remove minority samples.
C. Duplicate majority class.
D. Ignore the problem.

A60 — Answer: A

Correct: A.

`weightCol` or `class_weight` upweights minority class.

B, C make the problem worse.

Q61 — Metric Selection

Large errors are especially costly in a regression task. Best metric?

A. RMSE.
B. MAE.
C. Accuracy.
D. F1.

A61 — Answer: A

Correct: A.

RMSE squares errors $\rightarrow$ penalizes large deviations.

MAE treats all errors linearly.

Q62 — Spark ML Persistence

How is a fitted Spark ML model saved?

A. `model.write().overwrite().save(path)`.
B. `mlflow.log_artifact()`.
C. `spark.sql("SAVE")`.
D. `dbutils.fs.put()`.

A62 — Answer: A

Correct: A.

Spark MLModel has `save()`/`write()` — saves directory with metadata + Parquet.

`log_artifact()` logs to MLflow — different mechanism.

Q63 — CrossValidator Grid

How is the hyperparameter grid specified in Spark ML `CrossValidator`?

A. `ParamGridBuilder`.
B. Python dict only.
C. CSV.
D. SQL query.

A63 — Answer: A

Correct: A.

`ParamGridBuilder().addGrid(...).build()`.

Q64 — Evaluator Choice

Which evaluator pairs with `CrossValidator` for regression?

A. `RegressionEvaluator`.
B. `BinaryClassificationEvaluator`.
C. `MulticlassClassificationEvaluator`.
D. `ClusteringEvaluator`.

A64 — Answer: A

Correct: A.

`RegressionEvaluator` supports RMSE, MAE, R^2.

Classification evaluators expect label/score structure.

Q65 — Hyperopt Objective

What must the Hyperopt objective return?

A. Scalar loss to minimize.
B. DataFrame.
C. Trained model only.
D. String.

A65 — Answer: A

Correct: A.

`fmin(fn, space, algo, max_evals)` → returns lowest scalar loss.

To maximize, return `-metric`.

Q66 — Hyperopt Space

Uniform distribution over a range?

A. `hp.uniform('x', low, high)`.
B. `hp.choice('x', [a,b])`.
C. `hp.quniform('x', low, high, q)`.
D. `hp.loguniform('x', low, high)`.

A66 — Answer: A

Correct: A.

`hp.uniform` = continuous uniform.

`quniform` = quantized, `loguniform` = log-scale.

Q67 — Logistic Regression Class

Which Spark ML class trains binary logistic regression?

A. `LogisticRegression`.
B. `LinearRegression`.
C. `DecisionTreeClassifier`.
D. `RandomForestRegressor`.

A67 — Answer: A

Correct: A.

Q68 — Linear Regression Class

Which Spark ML class trains linear regression?

A. `LinearRegression`.
B. `LogisticRegression`.
C. `DecisionTreeClassifier`.
D. `NaiveBayes`.

A68 — Answer: A

Correct: A.

Q69 — Feature Importance

How is feature importance accessed for a tree model?

A. `model.featureImportances`.
B. `model.coefficients`.
C. `model.intercept`.
D. `model.summary`.

A69 — Answer: A

Correct: A.

Tree-based models expose `featureImportances` (vector).

Linear models use `coefficients`.

Q70 — Linear Coefficients

How are coefficients accessed for Spark ML linear regression?

A. `model.coefficients`.
B. `model.featureImportances`.
C. `model.intercept` only.
D. `model.summary` only.

A70 — Answer: A

Correct: A.

Q71 — StringIndexer Invalid

Default behavior for unseen label at transform time?

A. Throws error unless `handleInvalid` set.
B. Silently ignores row.
C. Maps to index 0.
D. Maps to most frequent.

A71 — Answer: A

Correct: A.

Default = `"error"`.

`handleInvalid="skip"` drops row; `"keep"` → index = numLabels.

Q72 — OneHotEncoder Input

What input does `OneHotEncoder` expect?

A. Numeric index column (from StringIndexer).
B. Raw string column.
C. Continuous feature.
D. Vector column.

A72 — Answer: A

Correct: A.

`StringIndexer` → `OneHotEncoder` → `VectorAssembler`.

Q73 — VectorAssembler Input Types

What column types can `VectorAssembler` accept?

A. Numeric, boolean, and vector columns.
B. Only string columns.
C. Only categorical columns.
D. Only binary columns.

A73 — Answer: A

Q74 — Pipeline Fit Output

What does fitting a `Pipeline` produce?

A. `PipelineModel`.
B. Raw DataFrame.
C. Cluster.
D. MLflow experiment.

A74 — Answer: A

Q75 — Model Save Format

What format does Spark ML use to save models?

A. Directory with metadata and Parquet.
B. Single CSV.
C. Pickle only.
D. JSON only.

A75 — Answer: A

Q76 — MLflow Model Flavor

What is an MLflow model flavor?

A. Convention for saving/loading models from different libraries.
B. Type of cluster.
C. Visualization tool.
D. Data source.

A76 — Answer: A

Q77 — Model Signature

Purpose of a model signature?

A. Defines expected input/output schema.
B. Encrypts the model.
C. Compresses the model.
D. Deletes the model.

A77 — Answer: A

Q78 — StandardScaler Params

Which parameters does `StandardScaler` use?

A. Mean and std.
B. Min and max.
C. Median and IQR.
D. Mode and variance.

A78 — Answer: A

Q79 — MinMaxScaler Range

Default scaling range of `MinMaxScaler`?

A. $[0, 1]$.
B. $[-1, 1]$.
C. $[0, 100]$.
D. Mean 0, var 1.

A79 — Answer: A

Q80 — Imputer Strategies

Which strategies does Spark ML `Imputer` support?

A. Mean, median, mode.
B. Only mean.
C. Only zero.
D. Only median.

A80 — Answer: A

Q81 — CV numFolds

What does `numFolds` control in `CrossValidator`?

A. Number of k folds.
B. Number of models to deploy.
C. Number of features to select.
D. Number of clusters.

A81 — Answer: A

Q82 — trainRatio

What does `trainRatio` control in `TrainValidationSplit`?

A. Fraction used for training.
B. Number of folds.
C. Number of trials.
D. Learning rate.

A82 — Answer: A

Q83 — DecisionTreeClassifier

Which class trains a decision tree classifier?

A. `DecisionTreeClassifier`.
B. `LinearRegression`.
C. `LogisticRegression`.
D. `KMeans`.

A83 — Answer: A

Q84 — RandomForestClassifier

Which class trains a random forest classifier?

A. `RandomForestClassifier`.
B. `DecisionTreeRegressor`.
C. `LinearSVC`.
D. `NaiveBayes`.

A84 — Answer: A

Q85 — GBT

Which class trains a gradient-boosted tree classifier?

A. `GBTClassifier`.
B. `RandomForestClassifier`.
C. `DecisionTreeClassifier`.
D. `LogisticRegression`.

A85 — Answer: A

Q86 — KMeans

Which class performs k-means clustering?

A. `KMeans`.

B. `LogisticRegression`.

C. `LinearRegression`.

D. `RandomForestClassifier`.

A86 — Answer: A

Q87 — Binary Metric

Which metric is available in `BinaryClassificationEvaluator`?

A. Area under ROC.
B. RMSE.
C. MAE.
D. R^2.

A87 — Answer: A

Q88 — Regression Metric

Which metric is available in `RegressionEvaluator`?

A. RMSE.
B. Precision.
C. Recall.
D. F1.

A88 — Answer: A

Q89 — Multiclass Metric

Which metric is available in `MulticlassClassificationEvaluator`?

A. F1.

B. RMSE.

C. MAE.

D. R^2.

A89 — Answer: A

Q90 — FeatureHasher

Purpose of `FeatureHasher`?

A. Hash categorical features to fixed-length vector without dictionary.
B. Impute missing values.
C. Scale features.
D. Split data.

A90 — Answer: A

No vocabulary needed $\rightarrow$ constant memory.

Downside: hash collisions possible.

Q91 — SQLTransformer

Purpose of `SQLTransformer`?

A. Apply SQL expression to transform within a Pipeline.
B. Train model.
C. Evaluate model.
D. Deploy model.

A91 — Answer: A

Q92 — VectorSlicer

Purpose of `VectorSlicer`?

A. Select subset of features from a vector column.
B. Combine columns into vector.
C. Scale features.
D. Impute missing.

A92 — Answer: A

Q93 — PolynomialExpansion

Purpose of `PolynomialExpansion`?

A. Generate polynomial and interaction features.
B. Encode categorical.
C. Scale features.
D. Split data.

A93 — Answer: A

Q94 — Binarizer

Purpose of `Binarizer`?

A. Threshold a continuous feature into binary.
B. Encode categorical.
C. Scale features.
D. Impute missing.

A94 — Answer: A

Q95 — Summary Statistics

Which command computes summary statistics on a Spark DataFrame?

A. `df.summary()`.
B. `df.show()`.
C. `df.printSchema()`.
D. `df.count()`.

A95 — Answer: A

`df.summary()` → count/mean/std/min/quartiles/max for numeric columns.

`describe()` gives a simpler set.

Q96 — Std-dev Outlier Removal

How to remove outliers based on standard deviation?

A. Filter values beyond $k\sigma$ from mean.
B. Remove all above median.
C. Remove top 50%.
D. `dropDuplicates()`.

A96 — Answer: A

Common choice: $|x - \mu| > 3\sigma \rightarrow$ drop.

Assumes roughly normal distribution.

Q97 — IQR Method

IQR outlier detection rule?

A. Below Q1 or above Q3.

B. $< Q1 - 1.5\,IQR$ or $> Q3 + 1.5\,IQR$.

C. Beyond mean.

D. $z > 1$.

A97 — Answer: B

Standard Tukey fence: $Q1 - 1.5\,IQR$ to $Q3 + 1.5\,IQR$.

More robust than z-score to non-normal data.

Q98 — Pandas API Refactor

Pandas notebook needs to scale to big data with minimal refactor. Best approach?

A. Rewrite in Scala.
B. Refactor to pandas API on Spark.
C. Use only Spark SQL.
D. Sample to memory.

A98 — Answer: B

pandas API on Spark mirrors pandas syntax on distributed data.

Sampling loses information; Scala rewrite is expensive.

Q99 — Two Continuous Features

Which visualization for comparing two continuous features?

A. Scatter plot.
B. Bar chart of counts.
C. Pie chart.
D. Box plot of one feature.

A99 — Answer: A

Scatter shows relationship/correlation between two continuous variables.

Q100 — Two Categorical Features

Which visualization for comparing two categorical features?

A. Grouped bar chart or heatmap.
B. Scatter plot.
C. Line chart.
D. Histogram.

A100 — Answer: A

Crosstab $\rightarrow$ grouped bar or heatmap makes comparison easy.

Exam Day Tips

Time budget: 1.9 min per question (48 Q / 90 min).

Watch for **"Select two"** — these appear frequently.

Kill time on traps: "accuracy" on imbalanced data, "scale before split", "means of serving" etc.

Priority domains: Databricks ML (38

Hands-on practice: MLflow + UC registry + Feature Store + Spark ML Pipeline.